ParamRF Module/Model refactor review

Branch: `feature/module-model-separation`

Scope

This change introduces a Parax-centred `pmrf.Module`, retains `pmrf.Model` as the RF-centred subclass, deprecates legacy `Model.build()`, and generalizes the fitting, optimization, and inference interfaces to operate on `Module` trees.

No existing RF model classes were migrated away from `build()` in this change. They remain compatible and now emit a `FutureWarning` when the legacy method is used.

Class organization

```
equinox.Module
    `-- pmrf.Module
          `-- pmrf.Model
```

`pmrf.Module` now owns the non-RF parameter-tree behavior:

- `name` and `metadata`
- `named_params()`
- `at()`
- `map()`
- generic `tied()` support
- representation helpers
- `is_module()` and module-wide parameter validation

`pmrf.Model` inherits that behavior and contains RF-specific functionality:

- port introspection
- S, A, Y, Z, and MNA response dispatch
- RF composition and transformations
- the `Wrapped` adapter for modules that unwrap to RF models
- scikit-rf conversion, plotting, and Touchstone export

The generic implementations were physically removed from `Model`; they are not duplicated or forwarded there.

Public API

The package root now exports:

```
pmrf.Module
pmrf.is_module
pmrf.Model
pmrf.is_model
```

Every `Model` is a `Module`, while a parameter container can inherit from `Module` without claiming to expose an RF response:

```
class Experiment(pmrf.Module):
    gain: pmrf.Param
    circuit: pmrf.Model
```

Generic `Tied` and `Probabilistic` wrappers live under `pmrf.modules`. Calling `.tied()` on an RF model returns `Wrapped(Tied(model))`, preserving the RF interface without making the parameter wrapper RF-aware.

`pmrf.models.Wrapped` is a minimal adapter model. It unwraps its contained module and delegates only RF-specific properties and methods: port count, primary domain/matrix, S/A/Y/Z/MNA evaluation, and topology expansion.

`Model.build()` deprecation

Calling an overridden `Model.build()` now emits a visible `FutureWarning`:

```
Model.build() is deprecated. Use a pmrf.Module to hold parameters and models,
with explicit methods returning RF models.
```

The warning is emitted for both direct calls and implicit compatibility paths such as port discovery, primary-domain dispatch, matrix evaluation, and circuit flattening. The method remains functional in this release.

Fitting and solver changes

The following APIs and their result generics now accept and preserve a `ModuleT`, rather than requiring `ModelT`:

- `pmrf.fitting.fit()`
- `fit_minimize()` and `fit_sample()`
- `fit_joint()` and `fit_sequential()`
- `pmrf.optimize.minimize()`
- `pmrf.infer.sample()`
- `FitResult`, `OptimizeResult`, and `InferResult`

This permits fitting arbitrary parameter-bearing containers when a callable predictor is supplied. Existing RF-model fitting and string feature aliases are unchanged.

Tests

New tests cover:

- `Model` inheriting from `Module`
- module parameter naming and optics
- generic module ties and free-parameter reporting
- tied and probabilistic RF modules evaluated through `Wrapped`
- RF composition and wrapper stacking through `Wrapped`
- fitting a non-RF `Module` using a callable predictor
- direct and implicit `Model.build()` warnings

Final test result:

```
265 passed, 32 warnings in 131.95s
```

The warnings comprise existing Equinox and convergence warnings plus the new expected `Model.build()` deprecation warnings from legacy test models.

Documentation changes

- Removed the custom composite-model example that recommended overriding `Model.build()`.
- Removed it from the examples index.
- Changed the parameter naming/manipulation container example to inherit from `pmrf.Module`.
- Removed `build()` from the recommended custom RF response methods.
- Updated the Parax unwrapping discussion to refer to modules and RF response methods.
- Updated the API index and core-primitives documentation for `Module`, generic wrappers, and the `Wrapped` adapter.
- Scanned the documentation for stale `Model.at/map/named_params`, old wrapper paths, and `build()` recommendations.

Files changed

Core implementation:

- `pmrf/modules/base.py` (new)
- `pmrf/modules/wrapped.py` (new)
- `pmrf/models/adapters/wrapped.py` (new)
- `pmrf/modules/__init__.py` (new)
- `pmrf/models/base.py`
- `pmrf/__init__.py`
- `pmrf/models/__init__.py`

Fitting and solving:

- `pmrf/fitting/minimize.py`
- `pmrf/fitting/sample.py`
- `pmrf/fitting/routers.py`
- `pmrf/fitting/result.py`
- `pmrf/optimize/minimize.py`
- `pmrf/optimize/result.py`
- `pmrf/infer/sample.py`
- `pmrf/infer/result.py`

Tests:

- `tests/test_module.py` (new)
- `tests/test_model.py`

Documentation:

- `docs/core_concepts/jax_overview.rst`
- `docs/examples/custom_parametric_models.rst`
- `docs/examples/parameter_naming_and_model_manipulation.rst`
- `docs/examples/index.rst`
- `docs/examples/custom_composite_models.rst` (removed)

Review points

1. Whether `FutureWarning` is the desired warning category and message.
2. Whether generic `Module.tied()` should remain part of this initial split or be deferred to a subsequent relationship API.
3. Whether low-level `optimize` and `infer` accepting `Module` is desirable in addition to the explicitly requested fitting layer.
4. Whether the removed composite example should later return as a `Module` assembly example with an explicit circuit-returning method.